

## Manual do Aluno — Flutter + Firebase

### Projeto: Aplicativo de Cadastro de Alunos

**Curso:** Técnico em Desenvolvimento de Sistemas

**Tecnologia:** Flutter + Firebase

**Nível:** Iniciante/Intermediário

**Objetivo:** Criar um aplicativo Flutter capaz de cadastrar, listar, editar e excluir alunos utilizando o Firebase como banco de dados.

**Importante:** Este manual é exclusivo para o aluno. **Não contém o gabarito nem o código completo pronto.** O objetivo é que você desenvolva o aplicativo seguindo as etapas e complete as partes solicitadas.

---

### 1. Objetivos da aula

Ao final da atividade, você deverá ser capaz de:

- Criar um projeto Flutter.
  - Entender a estrutura inicial de um projeto Flutter.
  - Criar um projeto no Firebase.
  - Conectar um aplicativo Flutter ao Firebase.
  - Configurar o Firebase para Android.
  - Instalar pacotes necessários.
  - Utilizar o Cloud Firestore.
  - Cadastrar informações.
  - Recuperar informações.
  - Exibir dados em uma lista.
  - Editar registros.
  - Excluir registros.
  - Trabalhar com Future, async e await.
  - Utilizar setState() para atualizar a interface.
- 

### 2. Projeto da aula

Você deverá desenvolver um aplicativo chamado:

## Cadastro de Alunos

O aplicativo deverá permitir cadastrar alunos contendo:

- Nome
- Idade
- Curso

Depois do cadastro, os alunos deverão aparecer em uma lista.

## Funcionalidades obrigatórias

Seu aplicativo deverá possuir:

| Funcionalidade          | Obrigatória |
|-------------------------|-------------|
| Cadastrar aluno         | ✓           |
| Listar alunos           | ✓           |
| Editar aluno            | ✓           |
| Excluir aluno           | ✓           |
| Banco de dados Firebase | ✓           |
| Interface organizada    | ✓           |
| Validação dos campos    | ✓           |

---

## 3. Estrutura inicial do Flutter

Antes de começar, vamos entender a estrutura básica.

Ao criar um projeto Flutter, você encontrará uma estrutura semelhante a:

```
meu_projeto/  
|  
├── android/  
├── ios/  
├── lib/  
|   └── main.dart
```

```
|
|
├── test/
├── pubspec.yaml
└── README.md
```

## Principais partes

### **lib/**

É onde ficará a maior parte do código Dart do aplicativo.

### **main.dart**

É o arquivo inicial da aplicação.

### **pubspec.yaml**

É utilizado para configurar o projeto e adicionar dependências.

### **android/**

Contém a configuração específica para Android.

---

## 4. 🚀 Criando o projeto Flutter

Abra o terminal do VS Code.

Execute:

```
flutter create cadastro_alunos
```

Entre na pasta:

```
cd cadastro_alunos
```

Abra o projeto no VS Code:

```
code .
```

Execute o projeto:

```
flutter run
```

### ✅ Verificação

Antes de continuar, confirme:

- O projeto foi criado.
- O VS Code abriu corretamente.

- O aplicativo executou.
  - O main.dart está funcionando.
- 

## 5. 🔥 Criando o projeto no Firebase

Abra o site do Firebase.

[Firebase Console](#)

Faça login com sua conta Google.

Clique em:

**Adicionar projeto**

Digite:

Cadastro de Alunos

ou outro nome que identifique seu projeto.

Siga as etapas apresentadas pelo Firebase.

---

## 6. 🤖 Adicionando o aplicativo Android

Dentro do projeto Firebase, procure a opção para adicionar um aplicativo.

Selecione:

**Android**

O Firebase solicitará o:

**Nome do pacote Android**

Você deverá descobrir o applicationId do seu projeto.

Abra:

android/app/build.gradle.kts

Procure algo parecido com:

applicationId = "com.example.cadastro\_alunos"

⚠️ **Copie exatamente o valor encontrado no seu projeto.**

Não invente outro nome.

---

## 7. Baixando o google-services.json

Depois de cadastrar o aplicativo Android no Firebase:

1. Baixe o arquivo:

google-services.json

2. Localize o arquivo no computador.
3. Coloque-o dentro da pasta:

android/app/

A estrutura deverá ficar semelhante a:

cadastro\_alunos/

|

└─ android/

|

└─ app/

|

└─ google-services.json

|

└─ build.gradle.kts

|

└─ src/

|

└─ lib/

|

└─ pubspec.yaml

### **Atenção**

O arquivo deve estar diretamente dentro de:

android/app/

e não em:

android/

nem:

lib/

---

## 8. Configurando o Android

Abra:

android/app/build.gradle.kts

Você encontrará uma estrutura semelhante a:

```
plugins {  
    id("com.android.application")  
    id("kotlin-android")  
    id("dev.flutter.flutter-gradle-plugin")  
}
```

A configuração do Firebase precisa ser adicionada de acordo com a configuração atual recomendada pelo Firebase/Flutter.

### Desafio

Consulte a documentação oficial e identifique onde o plugin do Google Services deve ser configurado.

[Firebase para Flutter — documentação oficial](#)

**Não copie configurações de versões antigas encontradas em vídeos aleatórios.** A configuração do Firebase pode mudar conforme as versões do Flutter, Gradle e plugins.

---

## 9. Instalando as dependências

Agora precisamos adicionar os pacotes Firebase ao projeto.

No terminal:

```
flutter pub add firebase_core
```

Depois:

```
flutter pub add cloud_firestore
```

Execute:

```
flutter pub get
```

### Para que servem?

| Pacote        | Função                 |
|---------------|------------------------|
| firebase_core | Inicializar o Firebase |

| Pacote          | Função                          |
|-----------------|---------------------------------|
| cloud_firestore | Trabalhar com o Cloud Firestore |

---

## 10. 🔥 Configurando o Firebase no Flutter

Agora você deverá inicializar o Firebase no aplicativo.

No arquivo:

lib/main.dart

Você precisará:

1. Importar o Firebase Core.
2. Transformar o main() em uma função assíncrona.
3. Inicializar o Flutter.
4. Inicializar o Firebase.
5. Executar o aplicativo.

### Estrutura esperada

Complete o código abaixo:

```
import 'package:flutter/material.dart';
import 'package:firebase_core/firebase_core.dart';

void main() async {

  // Complete aqui

  runApp(
    MaterialApp(
      debugShowCheckedModeBanner: false,
      home: TelaPrincipal(),
    ),
  );
}
```

}

### **Pense:**

Por que precisamos inicializar o Firebase antes de executar o aplicativo?

---

## 11. **Criando o Firestore**

No Firebase Console, procure:

### **Firestore Database**

Clique em:

#### **Criar banco de dados**

Escolha a região disponível/recomendada para seu projeto.

Durante o desenvolvimento, o Firebase poderá apresentar opções de regras de segurança.

 **Atenção:** regras abertas são úteis apenas para testes controlados. Não deixe um aplicativo real com regras inseguras.

---

## 12. **Criando a coleção**

No Firestore, crie uma coleção chamada:

alunos

Cada documento representará um aluno.

A estrutura conceitual será:

alunos

|

└─ documento 1

|

└─ nome

|

└─ idade

|

└─ curso

|

└─ documento 2



```
|  ├── nome
|  ├── idade
|  └── curso
|
└── documento 3
    ├── nome
    ├── idade
    └── curso
```

---

### 13. Criando a interface

Sua tela deverá possuir:

-----  
CADASTRO DE ALUNOS

Nome  
[\_\_\_\_\_]

Idade  
[\_\_\_\_\_]

Curso  
[\_\_\_\_\_]

[ CADASTRAR ]

-----  
ALUNOS CADASTRADOS

 João

18 anos

Desenvolvimento de Sistemas

 Maria

17 anos

Informática

-----

---

#### 14. Criando os Controllers

Dentro do State da sua tela, crie controllers para os campos:

TextEditingController nomeController =

TextEditingController();

TextEditingController idadeController =

TextEditingController();

TextEditingController cursoController =

TextEditingController();

#### Pergunta

Por que utilizamos TextEditingController?

Escreva sua resposta:

---

---

---

#### 15. Conectando ao Firestore

Importe o Firestore:

```
import 'package:cloud_firestore/cloud_firestore.dart';
```

Agora você poderá acessar o banco através do Firestore.

A referência conceitual será:

```
FirebaseFirestore.instance
```

Para acessar a coleção:

```
FirebaseFirestore.instance
```

```
.collection("alunos");
```

---

## 16. + Criando o cadastro

Crie uma função chamada:

```
cadastrarAluno()
```

Ela deverá:

1. Ler o nome.
2. Ler a idade.
3. Ler o curso.
4. Verificar se os campos estão preenchidos.
5. Enviar os dados para o Firestore.
6. Limpar os campos.
7. Informar ao usuário que o cadastro foi realizado.

### Complete:

```
Future<void> cadastrarAluno() async {
```

```
    String nome = nomeController.text;
```

```
    String idade = idadeController.text;
```

```
    String curso = cursoController.text;
```

```
    if (nome.isEmpty ||
```

```
        idade.isEmpty ||
```

```
curso.isEmpty) {  
  
    // Faça uma mensagem de erro aqui  
  
    return;  
}  
  
// Envie os dados para a coleção "alunos"  
  
}
```

---

## 17. 🌿 Estrutura dos dados

Ao salvar o aluno, os dados deverão possuir esta estrutura:

nome → texto

idade → texto ou número

curso → texto

Exemplo:

nome: João

idade: 18

curso: Desenvolvimento de Sistemas

### ⚠️ Desafio

Tente descobrir como fazer com que a idade seja armazenada como **número** em vez de texto.

Dica:

`int.parse(...)`

---

## 18. 📋 Criando a lista de alunos

Agora seu aplicativo precisa consultar o Firestore.

Você poderá utilizar:

StreamBuilder

ou:

FutureBuilder

Para esta atividade, recomendamos que você tente utilizar:

StreamBuilder

### **Estrutura inicial**

```
StreamBuilder<QuerySnapshot>(  
  stream: FirebaseFirestore.instance  
    .collection("alunos")  
    .snapshots(),  
  
  builder: (context, snapshot) {  
  
    // Verifique se existem dados  
  
    // Crie a lista  
  
  },  
)
```

---

## **19. Entendendo o StreamBuilder**

O StreamBuilder permite acompanhar alterações nos dados.

Imagine:

Firebase

↓

novo aluno

↓

Stream



StreamBuilder



Flutter



Tela atualizada

Isso significa que, quando os dados mudarem, a interface poderá ser atualizada automaticamente.

---

## 20. Criando o ListView

Você deverá apresentar os alunos usando:

`ListView.builder()`

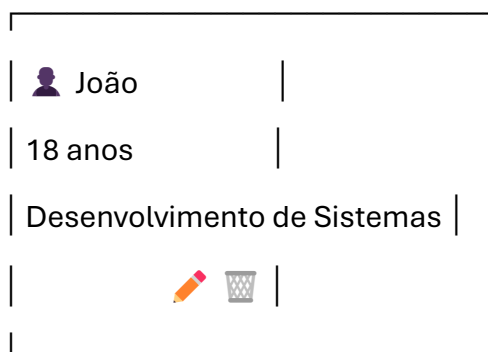
Cada aluno poderá ser apresentado utilizando:

Card

e:

ListTile

### Modelo visual



---

## 21. Editando um aluno

Cada aluno deverá possuir um botão de edição.

Exemplo:



Ao clicar:

1. Abra uma tela ou AlertDialog.
2. Mostre os dados atuais.
3. Permita modificar os dados.
4. Salve as alterações no Firebase.

Você deverá pesquisar e utilizar:

`update()`

### **Estrutura conceitual**

`FirebaseFirestore.instance`

```
.collection("alunos")  
  
.doc(id)  
  
.update({  
    // dados atualizados  
});
```

---

## **22. 🗑 Excluindo um aluno**

Adicione um botão:



Ao clicar:

1. Solicite confirmação.
2. Se o usuário confirmar, exclua o documento.
3. Atualize a lista.

Você deverá pesquisar o método:

`delete()`

### **Estrutura conceitual**

`FirebaseFirestore.instance`

```
.collection("alunos")
```

```
.doc(id)

.delete();
```

---

### 23. ⚠️ Confirmação de exclusão

Antes de excluir diretamente, faça uma pergunta:

Deseja realmente excluir este aluno?

[CANCELAR]    [EXCLUIR]

#### Desafio

Utilize:

`showDialog()`

para criar essa confirmação.

---

### 24. 🎨 Melhorando a interface

Depois que o CRUD estiver funcionando, melhore o visual.

Seu aplicativo deverá possuir:

#### AppBar

Cadastro de Alunos

#### Campos

- Ícone
- Label
- Borda arredondada

#### Botão

Use um botão visualmente destacado:

+ Cadastrar Aluno

#### Lista

Utilize Card para separar os alunos.

---



## 25. 🚧 Testando o cadastro

Faça os seguintes testes.

### Teste 1

Cadastre:

Nome: João

Idade: 18

Curso: Desenvolvimento de Sistemas

Verifique se aparece na lista.

---

### Teste 2

Cadastre outro aluno.

Verifique se os dois aparecem.

---

### Teste 3

Feche o aplicativo.

Abra novamente.

### Pergunta:

Os alunos continuam cadastrados?

Explique:

---

---

---

## 26. 🚧 Testando a edição

Escolha um aluno.

Altere:

Idade: 18

para:

Idade: 19

Salve.

Verifique no Firebase Console se o documento também foi alterado.

---

## 27. 🧪 Testando a exclusão

Escolha um aluno.

Clique no botão:



Confirme.

Depois abra o Firebase Console.

Verifique se o documento foi removido.

---

## 28. 🧠 Atividade 1 — Cadastro básico

Crie o aplicativo contendo:

- Campo Nome
- Campo Idade
- Campo Curso
- Botão Cadastrar
- Firebase conectado
- Coleção alunos
- Lista de alunos

### Critério

O aluno deve conseguir cadastrar pelo menos **3 alunos**.

---

## 29. 🧠 Atividade 2 — Validação

Modifique seu aplicativo para impedir:

- Nome vazio.
- Idade vazia.
- Curso vazio.

Mostre uma mensagem informando o problema.

### Teste

Clique em **Cadastrar** sem preencher nenhum campo.

O aplicativo deverá informar que os campos precisam ser preenchidos.

---

### 30. 🧠 Atividade 3 — Edição

Implemente a função de edição.

O usuário deverá conseguir:

Aluno

↓

Editar

↓

Alterar dados

↓

Salvar

↓

Firebase atualizado

### Requisito

Não crie um novo aluno ao editar.

Você deve alterar o documento existente.

---

### 31. 🧠 Atividade 4 — Exclusão

Adicione a exclusão.

O fluxo deverá ser:

Clique no 🗑️

↓

Confirmação

↓

SIM



Documento excluído



Lista atualizada

---

### 32. 🚀 Atividade 5 — Nível médio

Agora você deverá adicionar um novo campo:

#### E-mail

O cadastro deverá ficar:

Nome

Idade

Curso

E-mail

No Firebase:

nome

idade

curso

email

A lista também deverá mostrar o e-mail.

---

### 33. 🚀 Atividade 6 — Pesquisa

Adicione um campo:

🔍 Pesquisar aluno

O usuário deverá conseguir procurar alunos pelo nome.

#### Desafio

Faça a lista ser filtrada conforme o usuário digita.

---

34. 🚀 Atividade 7 — Contador

No topo da tela, mostre:

Total de alunos: 5

O número deverá mudar conforme os registros forem adicionados ou excluídos.

35. 🏆 Desafio final

Crie uma versão profissional do aplicativo.

Requisitos

Tela principal

-----

 MEUS ALUNOS

-----

🔍 Pesquisar...

12 alunos cadastrados

 João Silva

18 anos

Desenvolvimento de Sistemas

 Maria Souza

|                                                                                                                                                                     |  |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------|--|
| 17 anos                                                                                                                                                             |  |
| Informática                                                                                                                                                         |  |
|   |  |
|                                                                                                                                                                     |  |

+

---

### 36. 🌸 Checklist do projeto

Antes de entregar, confira:

#### Flutter

- Projeto executa.
- Não existem erros no código.
- Interface organizada.

#### Firebase

- Projeto criado.
- Aplicativo Android cadastrado.
- google-services.json configurado.
- Firebase inicializado.
- Firestore configurado.

#### CRUD

- Create — cadastrar.
- Read — listar.
- Update — editar.
- Delete — excluir.

#### Interface

- Campos organizados.
- Botões funcionando.

- Lista funcionando.
  - Mensagens de erro.
  - Confirmação de exclusão.
- 

### 37. 📁 Entrega

Ao terminar, entregue:

#### 1. Projeto Flutter

A pasta completa do projeto.

#### 2. Evidência do Firebase

Print mostrando a coleção:

alunos

e seus documentos.

#### 3. Evidência do aplicativo

Print da tela principal funcionando.

#### 4. Demonstração

Você deverá demonstrar ao professor:

1. Cadastrar aluno.
  2. Cadastrar segundo aluno.
  3. Listar alunos.
  4. Editar aluno.
  5. Excluir aluno.
  6. Mostrar os dados no Firebase Console.
- 

### 38. 🎯 Perguntas para responder

Responda no seu caderno ou documento.

1.

Qual é a função do Firebase em uma aplicação Flutter?

---

**2.**

Para que serve o pacote `firebase_core`?

---

**3.**

Para que serve o `cloud_firestore`?

---

**4.**

O que é uma coleção no Firestore?

---

**5.**

O que é um documento?

---

**6.**

Qual método é utilizado para inserir dados?

---

**7.**

Qual método é utilizado para atualizar dados?

---

**8.**

Qual método é utilizado para excluir dados?

---

**9.**

Qual é a diferença entre `setState()` e o Firestore?

---

**10.**

Por que os dados continuam disponíveis depois de fechar o aplicativo?

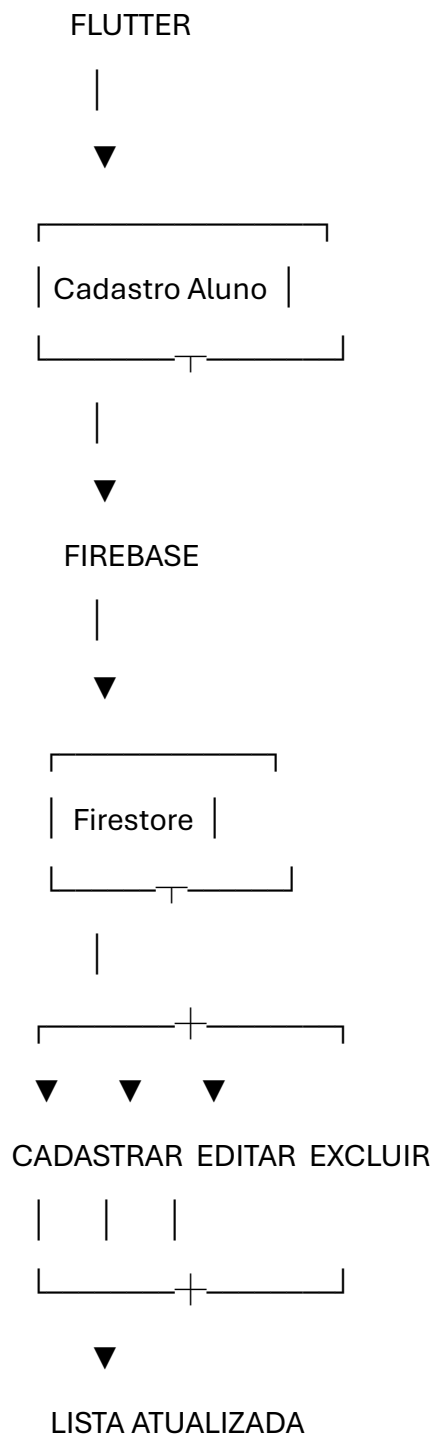
---

---



## 🔲 Desafio final para entrega

Seu aplicativo deverá cumprir o seguinte fluxo:



## ★ Regra principal

**Não basta fazer a tela funcionar.**

Você precisa conseguir explicar:

**"O que acontece desde o momento em que clico no botão até o dado aparecer no Firebase?"**

Se conseguir explicar esse fluxo e demonstrar o CRUD funcionando, você terá concluído o objetivo principal da aula.